



YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMICAL AND METALLURGY
DEPARTMENT OF MATHEMATICAL ENGINEERING

Final Project Work

ADVANCED CODE OBFUSCATION TECHNIQUES

Advisor: Prof.Dr. İbrahim EMİROĞLU

22058911 Izzat MAMMADZADA

Istanbul 2026

Contents

LIST OF FIGURES	v
Abstract	vi
1 Foundations of Code Obfuscation	1
1.1 Adversary Environment and Threat Modeling	1
1.2 Taxonomic Framework Overview.....	2
1.3 The Reverse Engineering Arsenal: Structural and Technical Mechanisms	3
1.4 Standardised Evaluation Framework.....	5
2 Establishing the Baseline	9
2.1 Grammatical Complexity and Parser Selection	9
2.2 Baseline Potency Quantification.....	11
2.3 Layered Security Framework Architecture	13
2.3.1 Hybrid AST-LLVM Transformation Pipeline	15
3 Function Merging (Architectural Collapse)	16
3.1 Identifier Renaming and Lexical Scrambling.....	16
3.2 Variable Declaration Separation and Context Hoisting.....	17
3.3 Scope Collision Resolution	18
3.4 Basic Block Identification and AST Partitioning	19
4 Control Flow Flattening (Structural Dismantling)	22
4.1 The Central Dispatcher Architecture	22
4.2 State Variable Management	23
4.3 Dismantling High-Level Loops	25
4.4 Multi-Level Flattening for Exception Handling	27
5 Opaque Predicates (Logical Complexity)	31
5.1 Principles of Bogus Control Flows	31
5.2 Vulnerabilities to Symbolic Execution.....	32
5.3 Designing "Bi-Opaque" Symbolic Predicates	34
5.4 Dead Code Insertion.....	35
6 Metamorphic Rewriting	37
6.1 Instruction Substitution Matrices	37
6.2 Code Transposition and Inline Assembly	38

6.3	Dynamic API and NTAPI Loading	40
6.4	Self-Modifying Code (SMC) via XOR Decryption.....	42
7	Conclusions & Future Work	45
	REFERENCES.....	47
	RESUME	48

LIST OF FIGURES

Figure 1.1: Symbolic Execution Analysis of Guarded Paths	4
Figure 1.2: Cyclomatic Complexity Comparison: Baseline vs. Flattened Logic	7
Figure 2.1: Normalized Baseline Logic (Three-Address Form & SSA Preparation)....	11
Figure 2.2: Lexical and Layout Obfuscation (Phase 1).....	12
Figure 2.3: Cumulative Phase 2 Data Transformation Logic	15
Figure 3.1: Lexical Scrambling Applied to Baseline.....	17
Figure 3.2: Context Hoisting of Nested Declarations.....	18
Figure 3.3: Collision Resolution and Memory Synchronization	19
Figure 3.4: AST Partitioning and Central Dispatcher Initialization	21
Figure 4.1: Central Dispatcher Initialization.....	22
Figure 4.2: Conditional State Transition	23
Figure 4.3: Secured Reference Assignment.....	23
Figure 4.4: Secured State Transitions (Relative Arithmetic).....	24
Figure 4.5: Secured State Transitions (Uniform Expressions)	25
Figure 4.6: While Loop Dismantling.....	26
Figure 4.7: For Loop Increment Processing	26
Figure 4.8: Unstructured Transfer Resolution.....	27
Figure 4.9: Multi-Level Flattening for Exception Handling	29
Figure 4.10: Cross-Level Unstructured Transfer Resolution	30
Figure 5.1: Bogus Flow Injection via Constantly True Guard.....	32
Figure 5.2: Bi-Opaque Predicate: Symbolic Memories (Type II False Positive)	33
Figure 5.3: Bi-Opaque Predicate: Symbolic Memories (Type II False Positive)	35
Figure 5.4: Bi-Opaque Predicate: Floating-Point Truncation (Type II)	35
Figure 5.5: Dead Code Payload in a Type I Bi-Opaque Guard.....	36
Figure 5.6: Hardware-Level Garbage Instruction Padding.....	36
Figure 6.1: XOR Equivalent	37
Figure 6.2: Randomized Matrix Substitution.....	38
Figure 6.3: Micro-level Transposition via Unconditional Jumps.....	38
Figure 6.4: Disassembly Disruption via Rogue Bytes	39
Figure 6.5: Stack Frame Diversification via Inline Assembly.....	39
Figure 6.6: Machine-code Substitution via Data Directives	40
Figure 6.7: Dynamic API Resolution and String Decryption.....	41
Figure 6.8: NTAPI Structure Initialization and Manual Error Handling	41
Figure 6.9: Direct Syscall Assembly Stub	42
Figure 6.10: DEP Bypass via Virtual Memory Allocation.....	43
Figure 6.11: SMC Inline Assembly: EIP Resolution and In-Place XOR Mutation	44

Abstract

This thesis details the design and implementation of a cumulative, four-phase software obfuscation architecture engineered to defend C++ applications against Man-At-The-End (MATE) adversaries. Modern reverse engineering toolchains, including static decompilers and dynamic symbolic execution engines, systematically bypass isolated software protection mechanisms. To counter these threats, this research deploys a hybrid transformation pipeline operating across both the Abstract Syntax Tree and the LLVM Intermediate Representation. The framework systematically degrades analytical heuristics through lexical scrambling and context hoisting, control flow flattening via state-driven dispatchers, the injection of bi-opaque predicates, and metamorphic machine-code rewriting utilizing dynamic Native API resolution and Self-Modifying Code. By dismantling the hierarchical execution graph and mutating the physical binary signature at runtime, the architecture non-linearly increases McCabe cyclomatic complexity and induces state-space ambiguity within automated theorem provers. The resulting implementation provides a mathematically defensible defense paradigm that maximizes the temporal and computational costs of reverse engineering while preserving deterministic execution semantics.

1 Foundations of Code Obfuscation

The following section establishes the theoretical framework of the research, defining the scope of the software protection domain and the adversarial MATE threat model that governs the design of our obfuscation pipeline.

1.1 Adversary Environment and Threat Modeling

The Man-At-The-End (MATE) threat model necessitates a shift from peripheral security to intrinsic software resilience. In this paradigm, the adversary maintains persistent, unrestricted control over the execution environment. This environment nullifies traditional perimeter defenses, as the attacker possesses read/write access to system memory, kernel-level execution control, and the ability to manipulate I/O streams.

Adversaries leverage the white-box nature of the MATE environment to bypass standard protection mechanisms through:

- **Static Decompilation:** Adversaries utilize advanced static analysis tools to reconstruct high-level source logic from machine code. This exposure facilitates the identification of critical algorithms, licensing logic, and proprietary data structures.
- **Runtime Memory Analysis:** The ability to inspect memory addresses during execution enables the extraction of transient assets, including decrypted buffers and secret keys, which are otherwise protected in non-MATE environments.
- **Dynamic Instrumentation:** Through frameworks like Frida or custom debuggers, attackers hijack control flow, perform function hooking, and bypass conditional checks in real-time, regardless of the obfuscation applied to static binary representations.

Defensive architectures in MATE scenarios prioritize increasing the temporal and computational costs of reverse engineering rather than achieving absolute security. Mitigation strategies focus on three core pillars:

1. **Computational Complexity Elevation:** Techniques aim to degrade the effectiveness of static and dynamic analysis tools. By increasing the depth and breadth of the call graph and introducing non-deterministic execution paths, the effort required to reconstruct source logic scales non-linearly with the complexity of the obfuscation layers.
2. **Structural Integrity and Diversification:** Implementing N-Version Obfuscation (NVO) creates distinct, functionally equivalent binary variants. This diversification prevents the development of universal exploits or patching scripts,

forcing the adversary to re-analyze the application for each unique deployment instance.

3. **Cryptographic Hardening:** Standard cryptographic primitives are vulnerable to key-scraping. White-Box Cryptography (WBC) integrates cryptographic constants and intermediate transformation states directly into the program logic, complicating the isolation of keys from the execution flow.

The subsequent sections detail the implementation of these techniques, starting with baseline logic and progressively applying cumulative obfuscation layers.

1.2 Taxonomic Framework Overview

The obfuscation framework implements a four-phase, cumulative transformation pipeline applied sequentially to the baseline C++ architecture. Each consecutive phase operates on the output of the previous layer, increasing the complexity of automated analysis and reverse engineering.

Phase 1: Function Merging and Lexical Scrambling The initial phase targets the visual and structural representation of the source code while establishing the prerequisite architecture for subsequent structural collapse. The obfuscator extracts a comprehensive symbol table and replaces human-readable identifiers with non-deterministic cryptographic hashes based on Unified Symbol Resolution strings. Simultaneously, the engine executes context hoisting, systematically isolating variable declarations from their underlying logic and moving them to the global function scope. This process degrades readability while preventing spatial visibility errors during later transformations.

Phase 2: Control Flow Flattening The second layer executes Control Flow Flattening to dismantle the structural logic of the execution path. Applied to the merged baseline, this layer reduces hierarchical control-flow structure of the C++ function. Native control structures such as loops and conditional branches are dissected into discrete basic blocks. These isolated blocks are subsequently encapsulated within a centralized switch statement governed by a dynamically updated state variable. This architectural collapse displaces static control flow dependencies, forcing execution to rely entirely on dynamic data-flow routing and increasing the difficulty of static analysis.

Phase 3: Opaque Predicates and Logical Complexity The third layer introduces logical traps into the flattened architecture to penalize automated analysis tools and symbolic execution engines. The compiler injects bogus control flows and dead-code payloads bounded by computationally irreducible mathematical guards. The architecture leverages traditional numerical invariants alongside advanced bi-opaque predicates that exploit pointer aliasing. This integration increases the complexity of decompiler analysis by inducing increased path complexity and state-space ambiguity.

Phase 4: Metamorphic Rewriting and Dynamic Execution The final transformation layer operates strictly at the machine-code level to mutate the physical binary signature and evade dynamic heuristic endpoint monitors. This layer executes instruction substitution and reorders functional basic blocks via code transposition. To decouple the application from static sandbox observation, the engine implements dynamic Native API resolution alongside Self-Modifying Code stubs. These stubs may modify selected code regions during runtime execution.

1.3 The Reverse Engineering Arsenal: Structural and Technical Mechanisms

Man-At-The-End (MATE) adversaries deploy specialized technical mechanisms to deconstruct, analyze, and manipulate compiled C++ applications. These toolsets systematically target the transformation layers applied during the compilation pipeline.

1. Static Translation: Disassemblers and Decompilers

Static translation tools reconstruct human-readable source code from compiled machine binaries, such as ELF or PE formats.

- **Disassemblers:** These tools translate binary machine code bytes into architecture-specific assembly language instructions. They map raw binary data streams to execution instructions. This process inherently discards preprocessor directives and high-level C++ metadata.
- **Decompilers:** Disassembler outputs are parsed by decompilers, such as IDA Pro, to define function boundaries and generate high-level pseudocode. Decompilers reconstruct implicit C++ execution mechanisms, including the recovery of the this pointer within object-oriented method invocations, directly decreasing the temporal cost of structural analysis.

2. Dynamic Execution Analysis: Debuggers and System Monitors

Dynamic analysis instruments evaluate the software state during execution, bypassing Phase 1 and Phase 2 static obfuscation layers by observing decrypted runtime states.

- **Debuggers:** Debuggers attach to active processes to trace execution state sequentially. They expose the live state of CPU registers and memory dumps, extracting data directly from active stack frames and heap allocations.
- **System Monitoring:** Interception tools monitor input/output channels between the target application and the Operating System kernel. By capturing file operations and API invocations, adversaries deduce program logic without direct memory inspection.
- **Dynamic Tampering and Injection:** Frameworks like Frida and Cycrypt inject external payloads directly into the active memory space of the target process. This

capability permits real-time manipulation of the execution environment and control flow hijacking.

3. Advanced Program Analysis: Symbolic Execution Engines

Symbolic execution engines automate the deobfuscation process by programmatically solving structural restrictions, such as the opaque predicates introduced in Phase 3 control flow transformations.

- **Intermediate Language (IL) Lifting:** Engines initially translate native binary instructions into an Intermediate Language representation, such as VEX IR or LLVM IR, to normalize semantic behavior analysis.
- **Constraint Modeling:** The analysis engine traces control paths, generating mathematical constraint models based on branch preconditions. These models are structured in Conjunctive Normal Form (CNF) as $\lambda_1 \wedge \lambda_2 \wedge \dots \wedge \lambda_n$, where each clause defines a programmatic predicate.
- **SMT Solvers:** Satisfiability Modulo Theories (SMT) solvers compute the concrete input variables required to satisfy the CNF constraint models. The solvers mathematically deduce the exact test cases needed to bypass static invariants.

```
int execute_guarded_path(int input_x, int input_y) {  
    int execution_state = 0;  
  
    // SMT solvers evaluate the concrete inputs required to resolve  
    // this Phase 3 opaque predicate statically.  
    if ((7 * (input_x * input_x) - 1) != (input_y * input_y)) {  
        execution_state = 1;  
    } else {  
        execution_state = 0;  
    }  
  
    return execution_state;  
}
```

Figure 1.1: Symbolic Execution Analysis of Guarded Paths

4. Binary Diffing and Similarity Detection

Binary diffing methodologies map modified binaries back to their original baseline logic by calculating algorithmic similarities.

- **Syntax-Based Diffing:** Diffing algorithms utilize graph-theoretic approaches to compare Control Flow Graphs (CFGs) and Call Graphs. The engine searches for graph isomorphisms to establish structural matches, rendering Phase 1 lexical and layout obfuscations ineffective.
- **Semantics-Based Diffing:** Semantic engines construct symbolic formulas to define the exact Input-Output relationships of individual basic blocks. Theorem provers calculate the longest common subsequence of semantically equivalent basic blocks. This isolates functionally identical C++ logic despite heavy visual modifications, directly degrading the efficacy of Phase 3 instruction substitution and bogus control flow insertions.

1.4 Standardised Evaluation Framework

The structural evaluation of C++ obfuscation quality formally relies on a four-dimensional taxonomy: Potency, Resilience, Stealth, and Cost. This framework quantifies the efficacy of the cumulative transformation layers applied during compilation.

1. Potency (Complexity and Obscurity Metrics)

Potency measures the theoretical difficulty required for decompilers or human analysts to parse the obfuscated logic. The framework calculates this using structural graph analyses.

- **McCabe's Cyclomatic Complexity:** Calculates the number of linearly independent control paths through a Control Flow Graph (CFG) via the equation $e - n + 2$, where e represents edges and n represents vertices. Phase 3 control flow flattening induces a severe, measurable multiplication in this metric.
- **Halstead Complexity Measures:** Quantifies the vocabulary of the algorithm by tracking the total and distinct number of operands and operators in the C++ Abstract Syntax Tree (AST).
- **Knot Count:** Calculates intersecting control-flow edge crossings when all nodes in a CFG are arranged linearly and edges are drawn on a single side.
- **Graph Volume:** Tracks the absolute numerical expansion of Basic Blocks, CFG edges, and Call Graph edges post-transformation.

The following code demonstrates how Phase 3 Control Flow Flattening, combined with Phase 2 Constant Unfolding, radically increases McCabe's Cyclomatic Complexity on a simple comparative baseline.

// Baseline Logic (Phase 0):

```
// int validate(int input_val) {  
//   int result = 0;  
//   if (input_val > 5) { result = 1; } else { result = 0; }  
//   return result;  
// }
```

// Cumulative Phase 3 Transformed Logic:

```
int validate_obfuscated(int input_val) {  
    int result_val = 0;  
    int state_variable = 1;  
  
    while (state_variable != 0) {  
        switch (state_variable) {  
            case 1:  
                // Phase 2: Constant unfolding transforms the static '5' threshold  
                if (input_val > (15 - 10)) {  
                    state_variable = 2;  
                } else {  
                    state_variable = 3;  
                }  
                break;  
            case 2:  
                result_val = 1;  
                state_variable = 0;  
                break;  
            case 3:  
                result_val = 0;
```

```

        state_variable = 0;
        break;
    default:
        state_variable = 0;
        break;
}
}
return result_val;
}

```

Figure 1.2: Cyclomatic Complexity Comparison: Baseline vs. Flattened Logic

2. Resilience (Resistance to Automated Deobfuscation)

Resilience quantifies the failure rate of automated reverse-engineering engines processing the transformed binaries.

- **AST/Source Diffing:** Computes the structural deviation at the source level. Tools normalise the layout and execute strict string-matching algorithms to calculate a diff percentage. High deviation metrics indicate degraded syntax-rebuilding capabilities in standard decompilers.
- **Syntax-Based Binary Diffing:** Utilises graph-theoretic algorithms to search for sub-graph isomorphisms between baseline and obfuscated call graphs. Phase 3 structural alterations reliably yield low function match rates, degrading BinDiff heuristics.
- **Semantics-Based Binary Diffing:** Employs theorem provers and Intermediate Language lifting to construct symbolic formulas defining basic block I/O relationships. Resilience is validated when satisfiability solvers fail to mathematically correlate the transformed basic block with its baseline equivalent due to Phase 2 data proceduralization and Phase 3 dead-code insertions.

3. Stealth (Statistical and Behavioral Anomaly Avoidance)

Stealth measures the statistical deviation of the obfuscated code against benign software baselines to evade heuristic-based malware detection and dynamic sandboxing.

- **Instruction Distribution Profiling:** Statically categorises compiled instructions (arithmetic, logical, jump) and compares their percentage frequencies against standard, un-obfuscated benchmark applications. Phase 3 transformations must

balance instruction substitution to prevent generating an excessive density of jump operations.

- **Byte Entropy:** Calculates the randomness of the binary data segments. Phase 2 string encryption and Phase 4 virtualization artificially spike byte entropy. High structural entropy flags the binary to dynamic unpacker heuristics, requiring deliberate calibration.
- **Indicator of Compromise (IoC) Tracking:** Dynamically executes the compiled binary in sandboxed environments to monitor API hooks. High stealth successfully masks process enumeration and conceals sensitive NTAPI injections, preventing the sandbox from generating actionable memory process dumps.

4. Cost (Spatial and Temporal Overhead)

Cost evaluates the engineering penalty generated by the cumulative obfuscation pipeline. This dimension strictly governs deployability in resource-constrained environments.

- **Temporal Overhead:** Runtime profilers calculate the percentage of CPU execution time and latency injected into the execution path. Phase 4 virtualization and interpreter dispatches introduce the highest temporal penalty, significantly degrading real-time processing capabilities.
- **Spatial Overhead:** Measured by the node-count expansion in the C++ AST and the final byte-size expansion of the compiled .text segments in memory. Phase 3 control flow flattening intrinsically bloats AST boundaries, while Phase 4 N-version compilation geometrically expands the final binary storage requirements.

2 Establishing the Baseline

This section details the construction of the 'CryptoCore' baseline architecture, defining the foundational programmatic structure, initial parsing logic, and the pre-mutation potency metrics required for iterative comparative analysis.

2.1 Grammatical Complexity and Parser Selection

Target C++ compilation relies on parsing highly ambiguous grammar parameters, including the "most vexing parse" anomaly. Constructing a proprietary parser introduces severe engineering overhead. The obfuscation architecture relies on established compiler front-ends, specifically libclang, the C Intermediate Language (CIL) library, or the Columbus framework. These front-ends systematically parse language specifications and generate an analyzable Abstract Syntax Tree (AST).

AST Extraction and the CXCursor Mechanism

The libclang interface traverses the source code utilizing the CXCursor object. The CXCursor functions as a pointer to the underlying C++ compiler objects, mapping syntactic constructs to precise source coordinates. Native reliance on CXCursor imposes two architectural constraints:

- **Immutability:** The exposed objects are immutable, designed strictly for read-only IDE introspection. This read-only state prevents direct obfuscation mutations within the native Clang tree.
- **Data Abstraction:** API abstraction limits access to low-level primitive values, such as integer literals. The pipeline must bypass the C-wrapper to query the native C++ AST directly for literal extraction.

Constructing the Mutable Baseline Tree

The architecture bypasses immutability constraints by iterating over CXCursor outputs to map a proprietary, mutable AST into memory. This intermediate structure instantiates an AstNode object for every parsed element. Each node categorizes the syntactic type (e.g., IntLiteral, IfStmt, StringLiteral) and encapsulates the raw data and child pointers. This custom mutable tree forms the structural foundation necessary for the execution of the four cumulative transformation phases.

Preprocessing and Code Normalization

The mutable AST undergoes strict normalization before execution of the mutation engines. The C++ preprocessor resolves macro expansions and #include directives prior to parser analysis. Conditional compilation directives (#ifdef) are permanently stripped, binding the resulting AST to the specific compilation environment platform.

The normalizer processes the AST into three-address forms. This process strips switch statements and ternary operators, transforms high-level loops into tail-recursive invocations, and restricts global, array, and struct manipulations to explicit pointer arithmetic. The pipeline enforces Static Single Assignment (SSA) properties by renaming variables and injecting memory flush/reload operations to eliminate implicit state dependencies prior to structural dismantling.

The following code demonstrates the Phase 0 normalization applied to the baseline logic. This step dismantles ternary operators and enforces SSA parameters before Phase 1 lexical transformations occur.

// Baseline Target Logic (Pre-Normalization)

```
int calculate_state(int input_x, int input_y) {  
    int execution_state = 0;  
    execution_state = (input_x > input_y) ? (input_x * 2) : (input_y + 1);  
    return execution_state;  
}
```

// Normalized Baseline Logic (Three-Address Form & SSA Preparation)

// This normalized AST serves as the exact baseline for Phase 1-4 transformations.

```
int calculate_state_normalized(int input_x, int input_y) {  
    int temp_cmp = 0;  
    int temp_mult = 0;  
    int temp_add = 0;  
    int result_state = 0;  
    temp_cmp = (input_x > input_y);  
    if (temp_cmp != 0) {  
        temp_mult = input_x * 2;  
        result_state = temp_mult;  
    } else {  
        temp_add = input_y + 1;  
        result_state = temp_add;  
    }  
}
```



```

    }

    return result_state;
}

```

Figure 2.1: Normalized Baseline Logic (Three-Address Form & SSA Preparation)

2.2 Baseline Potency Quantification

Establishing the complexity of the target C++ algorithm requires mathematical analysis of the pre-obfuscation Control Flow Graph (CFG) and Call Graph. These metrics define the baseline potency against which subsequent transformation layers are evaluated.

- McCabe’s Cyclomatic Complexity ($v(G)$): Quantifies linearly independent control paths through the CFG using the equation $v(G) = e - n + 2p$, where e represents edges, n represents vertices, and p represents connected components.
- Knot Count: Calculates intersecting control-flow edge crossings when nodes are arranged linearly and edges map to a single side. This defines the baseline entanglement prior to Phase 3 control flow flattening.
- Harrison Metrics: Quantifies the nesting depth of C++ control structures to evaluate baseline hierarchical complexity.

Lexical and Structural Volume Profiling

Structural volume metrics quantify the physical and vocabulary expansion injected by obfuscation overhead.

- AST Node Size: The absolute integer count of nodes within the pre-obfuscation Abstract Syntax Tree generated by the compiler front-end.
- Halstead Complexity Measures: Evaluates algorithmic vocabulary by calculating the total and distinct counts of operators and operands within the AST.
- Component Volume: The exact integer count of Basic Blocks, CFG Edges, and Call Graph Edges in the un-obfuscated executable.
- Identifier Complexity: Measures the human-readable vocabulary of class, function, and variable symbols targeted for Phase 1 hashing.

Cost Constraint Baselines

Empirical performance baselines define the operational threshold prior to obfuscation, measuring the temporal and spatial cost penalties.

- Temporal Baseline: Execution profilers determine the raw CPU time and latency required for the baseline C++ algorithm to execute target paths.

- Spatial Baseline: The pre-obfuscation byte-size of compiled .text segments and the executable payload. This metric evaluates the size expansion induced by inline assembly and N-version compilation.

Phase 1: Lexical and Layout Transformation Implementation

The normalized baseline algorithm undergoes Phase 1 transformations. The compiler pipeline executes identifier scrambling, replacing human-readable symbols with truncated cryptographic hashes. Symbol stripping removes comments, formatting, and debug metadata, degrading the baseline identifier complexity for static analysis tools without altering the AST execution semantics.

// Cumulative Phase 1 Transformed Logic: Lexical and Layout Obfuscation

// Applied to the normalized Phase 0 baseline. All identifiers are mapped to truncated hashes.

```
int f_8b2d1c(int v_3a9f2e, int v_7c4b1a) {
    int v_9e5d2f = 0;
    int v_1a8c3b = 0;
    int v_6d2f4e = 0;
    int v_5b9a1c = 0;
    v_9e5d2f = (v_3a9f2e > v_7c4b1a);
    if (v_9e5d2f != 0) {
        v_1a8c3b = v_3a9f2e * 2;
        v_5b9a1c = v_1a8c3b;
    } else {
        v_6d2f4e = v_7c4b1a + 1;
        v_5b9a1c = v_6d2f4e;
    }
    return v_5b9a1c;
}
```

Figure 2.2: Lexical and Layout Obfuscation (Phase 1)

2.3 Layered Security Framework Architecture

The layered obfuscation architecture partitions the compilation pipeline into four operational domains to enforce structural orthogonality. This distribution prevents localized transformations from neutralizing mechanisms applied at subsequent compilation phases.

- **Code-Element Layer:** Operates directly on the C++ Abstract Syntax Tree (AST). This layer executes localized syntactic mutations, including lexical identifier scrambling, data structure proceduralization, and basic block flattening.
- **Software-Component Layer:** Operates on the compiled binary executable or LLVM Intermediate Representation (IR). This layer applies macro-structural transformations, mapping imperative logic into translingual paradigms (e.g., Prolog), virtualizing native assembly into proprietary bytecode formats, and generating N-Version polymorphic binaries.
- **Cross-Component Layer:** Obfuscates inter-module execution boundaries. This layer masks dynamic Application Programming Interface (API) resolution and intercepts Native API (NTAPI) loading to degrade sandbox heuristics.
- **Application Layer:** Integrates environmental execution constraints. White-Box Cryptography (WBC) embeds decryption assets directly into the memory state, preventing the extraction of contiguous cryptographic keys.

Cumulative Technical Integration

Cumulative integration stacks orthogonal transformations to multiply reverse-engineering costs non-linearly. The output of the Code-Element layer functions as the direct input for the Software-Component layer, forcing the adversary to resolve multiple obfuscation paradigms sequentially.

- **Orthogonal Stacking:** Isolated control flow flattening is quantifiable and highly susceptible to Symbolic Execution engines. Stacking flattened control flows with Bi-Opaque predicates introduces parallel execution states and pointer aliasing ambiguities. This combination induces state-space explosion within Satisfiability Modulo Theories (SMT) solvers, degrading the efficacy of the automated deobfuscation engine.
- **Translingual and Binary Synergy:** Translingual obfuscation executes prior to virtualization, translating the C++ AST into a logic-based search-and-backtrack execution model. If an adversary successfully reverse-engineers the Phase 4 virtual machine interpreter, the recovered control flow remains structurally disjointed from standard imperative assembly, maintaining a high Halstead complexity metric.

- **Multi-Tiered Cryptographic Linkage:** Layered execution relies on segmented cryptographic dependencies. Frameworks like DOSP-AES isolate translation units using discrete AES key lengths (e.g., 128-bit for Name Obfuscation, 256-bit for Data Obfuscation). Execution requires the host architecture to fetch a unified master key from a remote server. The software parses this master string into specific subkeys to sequentially decrypt the layered states at runtime, preventing static offline analysis.

Phase 2: Data Transformation Implementation

The pipeline executes Phase 2 Data Transformations on the output generated by the Phase 1 Lexical pass. The compiler strips static integer assignments, replacing them with dynamically computed constant expressions using bitwise arithmetic and shifts. This degrades binary similarity matching algorithms.

// Cumulative Phase 2 Transformed Logic: Data Obfuscation

// Applied directly to the Phase 1 lexical baseline.

// Static literals (0, 1, 2) are substituted with dynamically computed bitwise constants.

```
int f_8b2d1c(int v_3a9f2e, int v_7c4b1a) {
    int v_9e5d2f = (0xAA ^ 0xAA);
    int v_1a8c3b = (0xAA ^ 0xAA);
    int v_6d2f4e = (0xAA ^ 0xAA);
    int v_5b9a1c = (0xAA ^ 0xAA);
    v_9e5d2f = (v_3a9f2e > v_7c4b1a);
    if (v_9e5d2f != (0xBB ^ 0xBB)) {
        v_1a8c3b = v_3a9f2e * (0x40 >> 5);
        v_5b9a1c = v_1a8c3b;
    } else {
        v_6d2f4e = v_7c4b1a + (0x2A ^ 0x2B);
        v_5b9a1c = v_6d2f4e;
    }
    return v_5b9a1c;
}
```

Figure 2.3: Cumulative Phase 2 Data Transformation Logic

2.3.1 Hybrid AST–LLVM Transformation Pipeline

The obfuscation framework adopts a hybrid transformation architecture combining source-level and compiler-level obfuscation stages. The initial transformation phases operate directly on the C++ Abstract Syntax Tree (AST) using libclang tooling, while later phases execute as custom LLVM backend passes over the Intermediate Representation (IR).

Phase 1 (Function Merging and Lexical Scrambling) and Phase 2 (Control Flow Flattening) are implemented as source-to-source transformations. These stages analyze and modify the mutable AST generated during the normalization process before regenerating transformed C++ source code.

Following source-level transformation, the generated code is compiled into LLVM Intermediate Representation. Phase 3 (Opaque Predicates and Logical Complexity) and Phase 4 (Metamorphic Rewriting and Dynamic Execution) operate on this IR through customized LLVM compiler passes. These passes manipulate low-level structures such as BasicBlock objects, control-flow edges, and IR instructions including AllocaInst, StoreInst, GetElementPtrInst, and ICmpInst.

This hybrid architecture combines the semantic visibility available at the AST level with the low-level transformation capabilities provided by LLVM IR. Source-level transformations restructure program semantics prior to compilation, while LLVM-based passes introduce machine-oriented obfuscation techniques that are impractical to implement reliably at the AST level.

3 Function Merging (Architectural Collapse)

This chapter initiates the first phase of the transformation pipeline, focusing on the destruction of modular function boundaries to create an inseparable, monolithic execution context.

3.1 Identifier Renaming and Lexical Scrambling

The lexical and layout transformation phase degrades human-readable semantics without modifying Abstract Syntax Tree (AST) execution logic. The obfuscation architecture hooks into the C++ compiler's parsing phase to extract a comprehensive symbol table, mapping identifiers to their underlying types, forms, and execution scopes. During AST traversal, the engine generates a Unified Symbol Resolution (USR) string for each node. This USR provides a unique cryptographic seed to ensure identifier renaming remains structurally consistent across disparate translation units.

Identifier mutation strictly obeys C++ compilation and linkage specifications to prevent undefined reference errors during binary assembly. The obfuscator parses visibility flags, systematically targeting `referencedLocally` while bypassing identifiers tagged with `isGlobal`. Identifiers exported via header files bypass the renaming algorithms to maintain cross-source dependencies. The linker entry point, explicitly designated by the `main()` function, is systematically excluded from lexical scrambling to ensure valid executable generation.

The obfuscator applies cryptographic generation algorithms to the isolated, safe-to-modify identifiers.

- MD5 USR Hashing: Computes the hash of the target identifier's USR string, truncating the output to generate a consistent, non-deterministic variable name.
- AES Substitution: Executes AES encryption using a randomized 128-bit key (K_{128}), substituting the original source identifier directly with the hex-encoded ciphertext.

The lexical scrambler algorithms physically compress the C++ source structure. The engine deletes whitespace, newline characters, and developer comments, collapsing the AST outputs into a contiguous byte stream where syntactically permissible. Following compilation, a secondary stripping pass targets the binary file. This pass deletes ELF symbol tables and debug metadata sections (`.line`, `.source`), eliminating the mapping between machine code instructions and the original C++ identifiers.

// Delta Snippet: Lexical Scrambling applied to Baseline

// Original declaration: int calculate_state(int input_x, int input_y)

int f_8b2d1c(int v_3a9f2e, int v_7c4b1a) {

```

int v_9e5d2f = 0;
int v_1a8c3b = 0;
// Remainder of variables replaced via truncated USR hashes

```

Figure 3.1: Lexical Scrambling Applied to Baseline

3.2 Variable Declaration Separation and Context Hoisting

Context hoisting executes as a mandatory preprocessing phase prior to the application of Control Flow Flattening (CFF). CFF algorithms partition the C++ function body into discrete basic blocks, assigning them to equivalent nesting levels within a unified switch statement bounded by a while loop. This architectural collapse strictly overrides the original lexical scope of the function. Locally scoped variables defined within nested blocks lose spatial visibility, inducing fatal compilation errors during the linking phase.

The Separation and Hoisting Algorithm

To prevent visibility errors, the preprocessing engine isolates variable declarations from their underlying logic and hoists them to the global function scope. The programmatic logic executes as follows:

1. **AST Traversal and Identification:** The engine parses the AST to identify variable declarations located beyond the absolute beginning of the function.
2. **Declaration Extraction:** The algorithm extracts the type and identifier of the flagged variables. These declarations are physically hoisted to the function apex.
3. **Initialization Mutation:** The engine strips the type definitions from the original inline locations. The initializations are mutated from active memory declarations into standard assignment statements, preserving the original execution sequence.

```

// Delta Snippet: Context Hoisting
int f_8b2d1c(int v_3a9f2e, int v_7c4b1a) {
    // Declarations extracted from nested blocks and hoisted to function apex
    int v_9e5d2f = (0xAA ^ 0xAA);
    int v_1a8c3b = (0xAA ^ 0xAA);
    float v_nested_float = 0.0f;
    // ... execution logic ...
}

```

```

if (v_3a9f2e == (0xBB ^ 0xBB)) {
    // Initialization mutated into assignment statement
    v_nested_float = 1.0f;
}

```

Figure 3.2: Context Hoisting of Nested Declarations

3.3 Scope Collision Resolution

The context hoisting phase induces artificial scope collisions. Control flow flattening displaces the original block-level hierarchy. When hoisting local variables from discrete nested scopes (e.g., independent loop iterators utilizing identical identifiers) into a unified function scope, identically named variables collide, triggering multiple-declaration syntax errors.

AST Detection and Cryptographic Renaming

The obfuscation engine tracks the lexical scope of each identifier during AST traversal. The algorithm enforces dynamic variable renaming upon detecting a collision during the hoisting phase. By hashing the USR string (which isolates spatial scope across translation units) using MD5 and truncating the output, the engine generates a mathematically unique identifier for the colliding variable.

Memory and Pointer Consistency Constraints

Cryptographic renaming of hoisted variables introduces semantic vulnerabilities regarding load-store consistency and pointer aliasing. Accessing renamed variables via memory pointers breaks program semantics through disjoint memory operations. The renaming algorithm implements a Flush and Reload mechanism to maintain C++ pointer logic integrity.

- **Points-To Analysis:** The engine executes a sound points-to analysis on the AST to identify addresses of local variables accessed via pointers.
- **Variable Flushing:** The compiler injects variable flushing operations prior to any read-from-pointer execution. This synchronizes the hoisted, renamed variable's state back to its original underlying memory address.
- **Variable Reloading:** The engine injects variable reloading operations immediately following a write-to-pointer execution. This synchronizes the symbolic variable with the active memory state.

// Delta Snippet: Collision Resolution and Memory Sync


```

// Organic collision resolved: Duplicate nested iterator cryptographically renamed
int iter_c4ca = 0;

// Alias synchronization via Flush/Reload mechanism
int* p_alias_8f2a = &v_1a8c3b;

// FLUSH: Sync hoisted variable to memory address prior to pointer use
*p_alias_8f2a = v_1a8c3b;

*p_alias_8f2a = *p_alias_8f2a + 1; // Pointer write operation

// RELOAD: Sync memory state back to hoisted variable immediately after write
v_1a8c3b = *p_alias_8f2a;

```

Figure 3.3: Collision Resolution and Memory Synchronization

3.4 Basic Block Identification and AST Partitioning

Control flow flattening requires the dissection of the target C++ function’s hierarchical body into discrete, equal-leveled basic blocks. Retaining standard language constructs natively precludes dispatcher execution. The obfuscation architecture algorithmically partitions the Abstract Syntax Tree (AST) to enforce geometric adjacency within a centralized switch mechanism.

The Partitioning Algorithm and Node Classification

Following the context hoisting phase, the engine invokes the `transform_block(block, entry, exit)` routine. This procedure parses the mutable AST, traversing the execution path to partition the function body into an indexed array of discrete chunks, logically defined as `block_parts[]`. During the partition phase, the algorithm strictly categorizes parsed C++ syntax nodes into two structural classifications:

- **Compound Statements:** Complex control structures that natively dictate branching logic, including `if`, `switch`, `while`, `do-while`, `for`, `try`, and nested block statements.
- **Non-Compound Sequences:** Linear execution instructions that do not branch the control flow, such as mathematical operations, pointer memory accesses, and standard function invocations.

State Variable Linkage and Flow Generation

State variable linkage enforces the execution sequence of the isolated basic blocks post-flattening. The `transform_block` algorithm calculates a specific `part_exit` state identifier

as it iterates through the `block_parts[]` array. If the analyzed part constitutes the terminal element in the block sequence, its `part_exit` directly inherits the overall exit state of the parent block. If the part constitutes a non-terminal element, the algorithm executes `unique_number()` to deterministically generate a unique target state. The computed `part_exit` state of the current block strictly functions as the entry state for the immediately subsequent block in the array, establishing logical linkage across the dismantled pieces.

Sub-Routing and Sequence Transformation

A dedicated routing mechanism evaluates the AST node type of the isolated block part, dispatching the chunk to specialized transformation subroutines (`transform_if`, `transform_while`, `transform_sequence`).

The `transform_sequence` routine processes sequences of linear, non-compound statements. The algorithm scans the basic block for unstructured control transfers, specifically `break` and `continue` keywords. Utilizing execution tracking stacks (breaks and continues stacks) managed during AST traversal, the engine strips these native C++ keywords. The architecture substitutes the stripped keywords with explicit assignment operations to the active control variable, hardcoding the execution flow back to the correct target state within the central dispatcher.

// Delta Snippet: AST Partitioning and Central Dispatcher Initialization

// Unstructured control transfers replaced with explicit state routing

// Phase 3 explicitly initializes the dispatcher state variable

unsigned int execution_state = 1;

// Transformed block array structurally flattened into a uniform hierarchy

while (execution_state != 0) {

switch (execution_state) {

case 1:

// Compound Statement block mapping to deterministically generated part_exit states

if (v_3a9f == (0xBB ^ 0xBB)) {

execution_state = 2;

} else {

execution_state = 3;

}

```

        break;
    case 2:
        // Non-Compound block executing isolated sequence
        v_9e5d_c4ca = 1.0f;
        // part_exit assigned to link with the subsequent block entry
        execution_state = 3;
        break;
    // ... additional block parts mapped to respective cases ...
    case 6:
        // Terminal part_exit state inheriting the parent block exit
        execution_state = 0;
        break;
    default:
        // Unstructured break/continue contingencies stripped and replaced with state
        assignments
        execution_state = 0;
        break;
    }
}

```

Figure 3.4: AST Partitioning and Central Dispatcher Initialization

4 Control Flow Flattening (Structural Dismantling)

Building upon the merged context, this section details the dismantling of the linear execution graph, replacing native C++ branching logic with a state-driven dispatcher model.

4.1 The Central Dispatcher Architecture

The primary structural objective of the Central Dispatcher Architecture is the total annihilation of a C++ function's hierarchical depth. The engine dismantles nested execution paths—such as hierarchical loops and compound branches—into a uniform, single-level hierarchy. This structural collapse displaces static control flow dependency, forcing execution to rely entirely on dynamic data flow. The spatial destruction of the code sequence effectively blinds static analysis tools, as the physical proximity of basic blocks no longer correlates to their execution sequence.

This transformation architecture fundamentally relies on a while-switch encapsulation framework, leveraging the basic block array generated during the AST partitioning phase.

// Phase 3 Dispatcher Initialization

```
unsigned int swVar = 1;
```

```
while (swVar != 0) {
```

```
    switch (swVar) {
```

```
        // Dispatched basic blocks...
```

Figure 4.1: Central Dispatcher Initialization

The architecture encapsulates the equal-leveled basic blocks within a centralized switch statement, which is subsequently bound by an infinite while loop. Execution flow is governed by a dedicated control state variable. The engine initializes this variable at the function apex, targeting the entry state of the initial basic block. The infinite loop continuously evaluates the state variable against a termination constraint. Inside the loop, the switch statement routes execution to the specific case label that corresponds to the active state integer.

At the terminus of every isolated basic block, the engine explicitly updates the control variable to the identifier of its logical successor, immediately terminating the case execution with a `break;` statement. This explicit update forces control back to the while loop apex, re-evaluating the switch logic to dispatch the subsequent block. For conditional execution paths, the engine dynamically updates the state variable based on the evaluated condition.

```

// Phase 3 Conditional State Transition
// Explicit state updates routing execution based on condition evaluation
if (v_9e5d != (0xCC ^ 0xCC)) {
    swVar = 4;
} else {
    swVar = 5;
}
break;

```

Figure 4.2: Conditional State Transition

However, a naive architectural implementation assigns literal integers to the state variable at the terminus of each block. This generates a vulnerability to local analysis, allowing decompilers to statically infer the exact successor path by directly observing the assigned integer.

To secure the dispatcher against local path-finding heuristics, the engine obfuscates the literal state transitions by structurally replacing direct assignments with dynamic computation logic. Advanced implementations utilize reference assignments, replacing static integers with relative arithmetic computations based on the active state value. The compiler strips explicit conditional branch assignments, substituting them with uniform mathematical expressions that evaluate dynamically to the successor state. The highest tier of implementation deploys one-way cryptographic functions to compute the successor identifier at runtime, strictly degrading automated static analysis capabilities.

```

// Phase 3 Secured Reference Assignment
// Direct assignment (swVar = 6) is replaced by a relative computation.
swVar = swVar + 2;
break;

```

Figure 4.3: Secured Reference Assignment

4.2 State Variable Management

Controlling execution flow via state variables, and securing the dispatcher by replacing direct state assignments with reference assignments or dynamic functions,. The architectural cornerstone of Control Flow Flattening (CFF) is the state variable, which replaces static execution flow with dynamic data-flow routing. The engine instantiates this variable at the apex of the dismantled function. It functions as the evaluation predicate for the central while-switch dispatcher, dictating the active execution block. At the terminus of every isolated basic block, the engine must explicitly update this variable to the identifier of the subsequent logical state, structurally forcing the dispatcher to route execution.

However, updating the state variable via direct literal assignments (e.g., `swVar = 3;`) introduces a critical structural vulnerability to local analysis. Decompilers trivially reconstruct the original Control Flow Graph (CFG) by isolating a basic block, parsing the hardcoded assignment, and directly mapping the transition.

To structurally decouple basic blocks from static path-finding heuristics, the engine obfuscates direct state assignments utilizing advanced dynamic computation techniques. The pipeline replaces absolute literal identifiers with reference assignments, updating the state variable relative to its active memory value. This technique hides the absolute destination address from static observation.

// Phase 3 Secured State Transitions: Reference Assignments

// The direct assignment (swVar = 4) is stripped and replaced with relative arithmetic.

// The absolute destination is hidden from static analysis.

`swVar = swVar + 1;`

`break;`

Figure 4.4: Secured State Transitions (Relative Arithmetic)

Furthermore, the engine dismantles explicit conditional branch assignments used for routing (e.g., `if (condition) swVar = 2; else swVar = 3;`). The pipeline substitutes these conditional statements with uniform, branchless mathematical expressions that evaluate dynamically to the correct successor state. Advanced architectures compute the successor identifier at runtime utilizing one-way cryptographic functions, severely limiting offline predictive capabilities.

// Phase 3 Secured State Transitions: Uniform Assignment Expressions

// The explicit conditional state routing is flattened into a branchless mathematical evaluation.

```

// If  $v\_9e5d \neq (0xCC \wedge 0xCC)$  (evaluates to 1),  $swVar$  becomes 4.
// If it evaluates to 0,  $swVar$  becomes 5.
 $swVar = (v\_9e5d \neq (0xCC \wedge 0xCC)) * 4 + (!(v\_9e5d \neq (0xCC \wedge 0xCC))) * 5;$ 
break;

```

Figure 4.5: Secured State Transitions (Uniform Expressions)

Unstructured control transfers, specifically break and continue keywords, introduce critical structural conflicts within the flattened hierarchy. Retaining native C++ unstructured transfers would inadvertently terminate or restart the centralized while dispatcher loop rather than the originally intended logic path. The AST parser deploys tracking structures during the compilation phase, specifically breaks and continues stacks, to map these unstructured keywords to their intended target states. The engine forcefully strips native C++ jump keywords from the source code. The pipeline replaces the stripped keywords with explicit state variable assignments, guided by the tracked mappings, ensuring precise routing back into the dispatcher framework without executing native compiler jumps.

4.3 Dismantling High-Level Loops

During Control Flow Flattening (CFF), isolating loop iterations into basic blocks is structurally complicated. Retaining native C++ loop constructs (while, do-while, for) within the flattened switch dispatcher corrupts compilation and program semantics, as detached loop heads fail to synthesize original execution logic. The Abstract Syntax Tree (AST) transformer structurally dismantles these high-level loops into explicit branching statements, dynamically routing execution via the centralized state variable.

The obfuscation pipeline deploys specialized transformation subroutines (transform_while, transform_do, transform_for) to process isolated loop logic into independent switch cases. The transform_while routine assigns a unique state identifier to the extracted loop body. The routine dismantles the while (predicate) head, substituting it with a discrete if-else block. This block evaluates the original condition, assigning the state variable to the body identifier on success, or routing to the loop's exit state on failure. The transform_do routine processes do-while loops by assigning test_entry and body_entry states. The entry execution unconditionally assigns the state variable to the body_entry to simulate the required initial execution pass. Following body termination, execution transitions to the test_entry case, routing back to the body or exiting based on the conditional evaluation.

// Phase 3 Loop Dismantling: While Loop Flattening

// The while predicate is dismantled into a discrete evaluation block.

// Execution relies entirely on the state variable to transition to the body.

case 7:

```
if (v_3a9f < 10) {  
    swVar = 8; // body_entry  
} else {  
    swVar = 9; // exit state  
}  
break;
```

Figure 4.6: While Loop Dismantling

The `transform_for` procedure executes a four-part partition due to the complex initialization, predicate, and increment parameters native to for loops. The routine establishes an initial entry state to evaluate the initialization logic, immediately transitioning to the `test_entry` state. The test state evaluates the loop condition, branching to the `body_entry` on success or routing to the overall exit state on failure. Following the execution of the isolated body block, the state variable transitions strictly to the `inc_entry` state. The increment block executes the required variable augmentation before explicitly routing the dispatcher back to the `test_entry` state for condition re-evaluation.

// Phase 3 Loop Dismantling: For Loop Increment Processing

// The increment logic is isolated into its own discrete basic block.

// Following incrementation, the state variable forces a return to the test_entry.

case 12: // inc_entry

```
v_3a9f = v_3a9f + 1;  
swVar = 10; // return to test_entry  
break;
```

Figure 4.7: For Loop Increment Processing

Dismantling high-level loops structurally orphans C++ unstructured control transfers. Native `break` or `continue` keywords retained within the flattened blocks will inadvertently terminate or reset the central dispatcher loop rather than controlling the original logical loop sequence. The AST parser manages breaks and continues tracking stacks during the partition phase to resolve this vulnerability.

The engine pushes the isolated loop's calculated exit state to the breaks stack and the relevant re-evaluation state (e.g., `inc_entry` for a for loop) to the continues stack. The `transform_sequence` routine actively strips native jump keywords from the AST, neutralizing native compiler directives. The engine replaces these keywords with explicit state variable assignments mapped from the tracking stacks. A standard `break;` command follows this assignment, terminating the active switch case and simulating the original unstructured transfer by cleanly routing the dispatcher state.

// Phase 3 Unstructured Transfer Resolution

// Native continue keywords are stripped to prevent dispatcher loop resets.

// Execution explicitly assigns the state variable from the continues stack.

case 15:

if (v_3a9f == 5) {

// Original logic: continue;

swVar = 12; // Assigned target mapped from continues stack (inc_entry)

break; // Terminate switch case, forcing dispatcher to route to case 12

}

// Standard execution block continues...

Figure 4.8: Unstructured Transfer Resolution

4.4 Multi-Level Flattening for Exception Handling

The structural integration of C++ exception handling introduces severe architectural constraints during Control Flow Flattening (CFF). The baseline flattening protocol, which dismantles hierarchical blocks into a unified switch dispatcher, cannot process try-catch segments. Dismantling and hoisting the contents of a try block into global dispatcher cases strips the lexical exception-handling boundaries. Any exceptions triggered during execution bypass the intended catch handlers, irreversibly corrupting program semantics.

To preserve lexical protections while enforcing structural obfuscation, the engine executes Multi-Level Flattening. This architecture flattens try blocks independently, constructing nested dispatcher structures.

The pipeline invokes the `transform_try` Abstract Syntax Tree (AST) routine to manage this structure. The engine initially generates a standard case within the primary central dispatcher to encapsulate the target exception structure. The routine bypasses standard

sequence partitioning and recursively invokes the `flatten_block` procedure directly on the try statement's body. This structural recursion generates an independent while-switch dispatcher nested directly within the lexical boundaries of the try block. The nested dispatcher executes utilizing its own localized state variable, physically disjoint from the primary control variable. The algorithm subsequently iterates through associated catch handlers, recursively applying the `flatten_block` procedure to each handler body, further compounding the multi-level complexity.

// Phase 3 Multi-Level Flattening: Exception Handling

// The try block executes a completely nested, independent dispatcher.

// The inner loop relies on localized state variable initialization (swVar2).

case 8: // Target case within the primary dispatcher (swVar1)

```

try {
    unsigned int swVar2 = 1; // Local state variable injection
    while (swVar2 != 0) {
        switch (swVar2) {
            case 1:
                // Isolated basic block inside the try structure
                v_3a9f = v_3a9f * 2;
                swVar2 = 2; // Route inner flow
                break;
            case 2:
                swVar2 = 0; // Terminate inner loop
                break;
        }
    }
} catch (...) {
    unsigned int swVar3 = 1; // Recursive flattening within catch handler
    while (swVar3 != 0) {
        // Catch block dispatcher logic...
    }
}

```

```

    }
}

swVar1 = 9; // Route outer flow post-exception block

break;

```

Figure 4.9: Multi-Level Flattening for Exception Handling

The multi-level nesting architecture introduces a severe control-flow anomaly regarding cross-level unstructured transfers. Unstructured transfers originating within a nested try block but targeting the outer parent loop (e.g., breaking out of a hierarchical loop surrounding the try block) fail under standard keyword stripping. Modifying the state variable alone traps execution within the immediate inner while loop.

To resolve cross-level flow routing, the AST parser deploys a levels tracking stack, logging (variable, label) tuples for every active dispatcher depth. Upon detecting a cross-level transfer, the engine executes targeted variable updates and explicit jumps. The engine targets the outer state variable (e.g., swVar1) mapped from the levels stack rather than updating the localized inner variable (e.g., swVar2). To physically break the inner loop's scope, the compiler injects an explicit C++ goto statement. While universally discouraged in standard development, the goto command is a structural requirement within this architecture to explicitly force execution out of the nested hierarchy and back to the labeled apex of the correct outer while dispatcher.

// Phase 3 Cross-Level Unstructured Transfer Resolution

// The execution must break the inner try-dispatcher (swVar2) to route the parent loop (swVar1).

OUTER_LOOP: // Labeled head mapped from the levels stack

```

while (swVar1 != 0) {
    switch (swVar1) {
        case 12:
            try {
                unsigned int swVar2 = 1;
                while (swVar2 != 0) {
                    switch (swVar2) {
                        case 1:

```

```
if (v_3a9f == 5) {  
    // Original logic: break; (targeting outer loop)  
    swVar1 = 15; // Target outer variable mapped via levels stack  
    goto OUTER_LOOP; // Force scope break  
}  
swVar2 = 2;  
break;
```

Figure 4.10: Cross-Level Unstructured Transfer Resolution

5 Opaque Predicates (Logical Complexity)

This chapter introduces logical traps into the flattened dispatcher, employing undecidable mathematical constructs to force the mapping of unexecutable paths by static analysis toolchains.

5.1 Principles of Bogus Control Flows

The fundamental architectural objective of injecting bogus control flows is the deliberate pollution of a program's Control Flow Graph (CFG) with unexecutable conditional branches and dead-code paths. These injected paths are strictly bypassed during runtime execution, preserving the baseline programmatic semantics. However, static analysis toolchains—including decompilers and static symbolic executors—are structurally forced to parse these paths. This fundamentally degrades their capacity to reconstruct functional logic.

Structurally, this technique serves to mathematically inflate program complexity metrics. The primary target is McCabe's Cyclomatic Complexity, computed as $e - n + 2p$. Bogus control flow injection artificially spikes this metric by simultaneously synthesizing new vertices (bogus basic blocks containing junk code) and edges (conditional jumps). Furthermore, injecting deeply nested fake loop structures artificially inflates structural nesting levels, severely degrading Harrison complexity metrics.

The compiler strictly enforces the unreachability of these bogus paths in compiled C++ by guarding the branching structures with opaque predicates. An opaque predicate is defined as a boolean mathematical expression whose truth value is predetermined at obfuscation-time but fundamentally intractable for an automated static analyzer to evaluate offline.

The pipeline executes C++ transformations by replacing linear baseline blocks with branching structures governed by three distinct opaque classifications. The first category utilizes Constantly True ($P \wedge T$) Predicates. The engine wraps legitimate C++ logic within an if statement guarded by a $P \wedge T$ expression. The else branch encapsulates the synthesized bogus control flow, which is mathematically isolated from execution.

// Phase 3 Bogus Flow Injection: $P \wedge T$ Predicate Guard

// The mathematical invariant ensures the if-branch (legitimate logic) always executes.

// The else-branch contains unreachable bogus code designed to confuse static analyzers.

```
if ((7 * (v_3a9f * v_3a9f) - 1) != (v_7c4b * v_7c4b)) {
```

```

// Legitimate execution logic
v_1a8c = v_3a9f * (0x40 >> 5);
} else {
// Bogus control flow (Junk code block)
v_1a8c = v_1a8c ^ 0xFFFF;
v_5b9a = v_1a8c * 42;
}

```

Figure 5.1: Bogus Flow Injection via Constantly True Guard

Conversely, the second category deploys Constantly False ($P \wedge F$) Predicates. A linear execution path is structurally interrupted by an if statement guarded by a $P \wedge F$ expression. This statement encapsulates the bogus code block, while legitimate execution strictly routes through the else branch or sequential flow. The final category implements Context-Dependent ($P \wedge ?$) Predicates. These involve sequences of correlated predicates whose discrete boolean evaluations vary dynamically based on the runtime context. However, strict mathematical correlation guarantees that the macro-behavior remains deterministic, ensuring the ultimate circumvention of the dummy paths.

Within the layered architecture, bogus control flows function as logical traps injected directly into the flattened basic blocks synthesized during the Control Flow Flattening (CFF) phase. While the centralized while-switch dispatcher destroys the original hierarchy, the bogus flows aggressively pollute the isolated cases with interconnected, unreachable basic blocks. If an adversary attempts to statically reconstruct the CFG via syntax-diffing, they encounter a densely populated maze of dummy branches mathematically indistinguishable from the legitimate execution path.

5.2 Vulnerabilities to Symbolic Execution

The structural flaw inherent to static opaque predicates is their reliance on deterministic mathematical logic, rendering them highly susceptible to automated Symbolic Execution (SE) engines such as KLEE and Triton. While numerical invariants effectively degrade standard decompiler heuristics, automated Satisfiability Modulo Theories (SMT) solvers trivially resolve these equations. SE engines dynamically flag guarding conditions as opaque predicates upon proving the mathematical constraints of a dummy branch are fundamentally unsatisfiable. This automated deduction permits the analysis engine to programmatically prune the dead code, successfully reverting the Control Flow Graph (CFG) to its baseline state.

We must understand how an SE-based adversary programmatically dissects execution states to engineer resilient counter-mechanisms. The SE engine initiates the deobfuscation pipeline by modeling the binary instructions utilizing Hoare Logic triples, defined mathematically as $\{S_1\}I\{S_2\}$, where S_1 functions as the prerequisite condition, I represents the execution instruction, and S_2 defines the resultant postcondition. The engine subsequently executes a backward trace from the target conditional branch, algorithmically calculating the Weakest Precondition (wp) necessary to achieve that specific state, defined as $S_0 = wp(I_0\{S_1\})$. The combined mathematical constraints are aggregated into a unified state model formatted exclusively in Conjunctive Normal Form (CNF), represented as $\lambda_1 \wedge \lambda_2 \wedge \dots \wedge \lambda_n$. This CNF model is processed by the SMT solver; if the solver mathematically deduces that no concrete variable inputs can satisfy the CNF model for the target branch, the engine confirms the dead-code status and bypasses the static opaque guard.

```
// Phase 3 Vulnerable Static Opaque Guard

// SMT solvers trivially prove that (a * (a + 1) % 2) always equals 0.

// The engine flags the false branch as bogus and prunes the CFG.

if (v_3a9f * (v_3a9f + 1) % 2 == 0) {
    swVar = 16; // Legitimate path
} else {
    // Flagged and pruned by the SE engine
    v_1a8c = v_1a8c ^ 0xFFFF;
}
```

Figure 5.2: Bi-Opaque Predicate: Symbolic Memories (Type II False Positive)

The proliferation of SMT-vulnerable constants severely compromises real-world obfuscation frameworks. If we embed complex, NP-hard structural obfuscation mechanisms but guard their execution using weak, static invariants, the SE engine trivially resolves the guard parameters, algorithmically reducing the targeted NP-hard problem back to polynomial-time complexity. We consistently observe highly vulnerable C++ constructs derived from academic literature, including constantly false invariants such as $((b - 2) * (b - 1) * b \% 6 \neq 0)$, which SMT solvers instantly prove fundamentally unsatisfiable. Furthermore, the default opaque predicate deployed by the industry-standard Obfuscator-LLVM framework evaluates the combined logic $\text{if } (x7 * (x7 - 1) \% 2 == 0 \parallel x8 < 10)$. An automated SE engine mathematically deduces that the left parameter

of the OR condition represents an absolute truth, instantly solving the entire boolean expression and neutralizing the control-flow flattening.

5.3 Designing "Bi-Opaque" Symbolic Predicates

We must architecturally evolve beyond traditional opaque predicates to counter the sophisticated capabilities of Symbolic Execution (SE) engines. "Bi-Opaque" predicates represent this structural evolution, engineered to induce both false negatives and false positives within the automated deobfuscation pipeline. Type I predicates function as traditional false negatives, deploying mathematically unsolvable conditions to guard injected dead code. Conversely, Type II predicates introduce a critical vulnerability to SE engines by causing false positives. This mechanism structurally alters legitimate, functional program predicates to resemble opaque constructs. This targeted manipulation tricks engines like KLEE or Angr into misclassifying valid execution paths as dead code, effectively forcing the analyzer to autonomously prune the functional algorithm.

To automatically integrate Bi-Opaque predicates during compilation, we implement a customized LLVM compiler pass, operating over frameworks such as Obfuscator-LLVM. This algorithm executes structural template injection at the Intermediate Representation (IR) layer. The compiler pass intercepts an icmp (integer comparison) instruction to isolate the active symbolic variable. The engine dynamically synthesizes complex LLVM IR data structures, utilizing AllocInst to allocate array memory, StoreInst to map the targeted variable, and GetElementPtrInst to enforce obfuscated pointer-based access. Finally, it deploys ICmpInst and BinaryOperator logic to weave the isolated variable into the designated Type I or Type II predicate guard.

```
// Phase 3 Bi-Opaque Predicate: Symbolic Memories (Type II False Positive)  
// Array initializations map symbolic variable 'j' through multiple index pointers.  
// If the SE engine fails the pointer analysis, it assumes 'i' cannot equal 1,  
// causing it to prune this legitimate execution path.  
int arr_a[5] = {2, 0, 4, 1, 3};  
int arr_b[5] = {9, 1, 8, 7, 6};  
int i = arr_b[arr_a[v_3a9f% 5]];  
  
if (i == 1 && v_3a9f == 7) {
```



```

    swVar = 17; // Legitimate path pruned by SMT solvers
}

```

Figure 5.3: Bi-Opaque Predicate: Symbolic Memories (Type II False Positive)

```

// Phase 3 Bi-Opaque Predicate: Floating-Point Truncation (Type II)
// SE engines evaluate this as a mathematical contradiction over real numbers.
// However, C++ float truncation satisfies the equation at runtime.
volatile float f = 0.000007f;
if (1024.0f + f == 1024.0f && f > 0.0f && v_3a9f == 7) {
    swVar = 18; // Executed at runtime, but statically pruned
}

```

Figure 5.4: Bi-Opaque Predicate: Floating-Point Truncation (Type II)

5.4 Dead Code Insertion

We implement dead code insertion to structurally bloat the compiled binary, altering its cryptographic hash and physical byte sequence to degrade signature-based detection heuristics. This technique operates as the direct physical payload for the unexecutable basic blocks synthesized during the prior Phase 3 Bi-Opaque transformations. While the Type I opaque predicates establish the logical barriers within the flattened while-switch dispatcher, dead code insertion provides the dense, non-functional material that populates these trapped cases. Our primary architectural objective is to force automated static analyzers to process massive volumes of irrelevant instructions without modifying the baseline execution semantics or inducing real-time performance penalties.

To execute this at the Abstract Syntax Tree (AST) level, our compiler pass injects non-functional arithmetic operations and dummy assignments directly into the mathematically isolated branches. We specifically utilize the hoisted variables established during the Phase 3 context hoisting sequence to ensure the garbage logic superficially resembles legitimate data flow, thereby confusing syntactic differs.

```

// Phase 3 Dead Code Payload within a Type I Bi-Opaque Guard
if (i == v_3a9f) {
    v_1a8c = (v_1a8c ^ 0xFFFF) * (v_5b9a % 3);
}

```

```

v_9e5d_c4ca = v_9e5d_c4ca + 3.14159f;
swVar = 99;
}

```

Figure 5.5: Dead Code Payload in a Type I Bi-Opaque Guard

Because the guarding Type I predicate mathematically guarantees this branch remains unexecutable at runtime, the mutated variables never corrupt the active program state. The injected junk code remains strictly resident in memory as a static analytical obstacle. Extending this paradigm below the C++ AST, our architecture operates directly on the emitted machine code by injecting continuous sequences of NOP instructions and explicit garbage bytes into the .text executable segments.

```

// Phase 3 Hardware-Level Garbage Instruction Padding
__asm__ volatile (
    "nop;\n\t"
    "nop;\n\t"
    "nop;\n\t"
);

```

Figure 5.6: Hardware-Level Garbage Instruction Padding

The strategic placement of explicit hardware padding bloats the binary footprint and severely disrupts the parsing algorithms of industry-standard disassemblers. When tools attempt linear or recursive descent traversal over the flattened basic blocks, they must decode these dense garbage instructions as if they were functional logic. This integration of unfeasible control flows and raw instruction bloat desynchronizes the static analyzer's internal instruction pointer, increasing disassembly complexity and analysis effort across subsequent legitimate blocks.

6 Metamorphic Rewriting

The final transformation layer descends to the machine-code level, where the binary signature is mutated via opcode substitution and dynamic self-modification to evade signature-based detection.

6.1 Instruction Substitution Matrices

Instruction substitution degrades static analysis heuristics at the physical machine-code level by altering the emitted byte sequence without modifying the execution semantics established during the prior Phase 3 dead code insertions. We apply this mechanism directly to the basic blocks and injected garbage payloads from the flattened dispatcher, mutating the underlying assembly representations of our hoisted variables. The compiler pass achieves semantic equivalence by intercepting standard, intuitive instruction emissions and substituting them with discrete logical equivalents.

// Phase 4 Instruction Substitution Delta

// The compiler intercepts standard memory initializations for hoisted variables.

// A standard variable zeroing is replaced with its logical XOR equivalent.

```
__asm__ volatile ("xor %%eax, %%eax" : "=a" (v_9e5d));
```

Figure 6.1: XOR Equivalent

This explicit hardware-level substitution yields a zeroed register while fundamentally scrambling the binary signature, ensuring that the previously injected NOP sleds and garbage instructions are further diversified against signature-based detection algorithms.

Baseline Matrix Branching Programs suffer from structural predictability due to their reliance on matrices populated exclusively by highly analyzable binary values of ones and zeros. Our compiler pass mitigates this vulnerability by executing randomized matrix substitution, replacing the baseline state structures with dynamically generated integer matrices. The obfuscator generates a sequence of $n+1$ random matrices denoted as RM_i alongside their exact mathematical inverses RM_i^{-1} . The original matrices are transformed via multiplication with these random pairs, forging heavily obscured equivalents utilizing the algebraic equation $RM_{\{0,0\}} = RM_0^{-1} \times M_{\{0,0\}} \times RM_1$.

// Phase 4 Randomized Matrix Substitution Delta

// The inverse pairs structurally cancel during sequential runtime multiplication.

*// $(RM_inv[i] * M_base * RM_next)$ resolves to the original M_base trajectory.*

```
current_state = multiply(multiply(RM_inv[i], M_base[i][input_bit]), RM_next[i+1]);
```

Figure 6.2: Randomized Matrix Substitution

During runtime evaluation, the continuous chain of multiplication causes the inverse pairs to mathematically cancel each other out across the execution sequence. This structural design ensures the randomized matrix program calculates the original C++ algorithm's correct outcome while permanently denying static analyzers access to the underlying boolean logic, successfully finalizing the architectural transformation of the application payload.

6.2 Code Transposition and Inline Assembly

Code transposition structurally degrades static analysis by destroying the spatial locality of the compiled binary without altering the underlying abstract syntax tree semantics. We implement this mechanism to randomly shuffle the linear sequence of original instructions and discrete basic blocks directly in memory. To maintain correct execution flow across these newly scattered memory segments, our compiler pass injects unconditional jump instructions that physically bridge the disjointed blocks.

```
// Micro-level transposition delta utilizing unconditional jumps  
  
goto block_c;  
  
block_a: v_1a8c ^= 0xFFFF; goto block_exit;  
  
block_c: v_9e5d = v_3a9f * 2; goto block_a;  
  
block_exit:
```

Figure 6.3: Micro-level Transposition via Unconditional Jumps

We execute this transposition across multiple granularity levels, shifting entire distinct subroutines at the macro-level to clutter the global layout, while simultaneously scrambling individual execution lines at the micro-level to heavily degrade intra-procedural control flow tracking.

Operating exclusively at the Intermediate Representation (LLVM IR) layer introduces a severe architectural limitation regarding raw byte manipulation, which is a strict requirement for advanced anti-disassembly techniques. To bypass this compiler abstraction, we bridge the IR and machine-code layers utilizing inline assembly. This integration logically relies on the unexecutable basic blocks synthesized during the prior Phase 3 Bi-Opaque transformations. Because the Type I opaque predicates mathematically guarantee their guarded paths will never execute, we utilize these isolated dead-code cases as safe structural containers for raw machine-code injection.

We inject rogue data bytes directly into these trap blocks to induce cascading disassembly failures within parsing tools like IDA Pro. At the C++ and LLVM IR level, we insert calls to the inline assembly engine that emit explicit hexadecimal data rather than valid operation mnemonics.

```
// Disassembly disruption delta targeting a Phase 3 dead-code block
if (i == v_3a9f) {
    __asm__ volatile (".long 0xdeadbeaf" : : : "memory");
}
```

Figure 6.4: Disassembly Disruption via Rogue Bytes

If the CPU attempts to execute this injected data as a machine instruction, the process will generate a fatal hardware exception. However, because we interleave this data strictly within the unfeasible branches governed by constantly false opaque conditions, runtime execution safely bypasses the trap. Disassemblers, unable to mathematically solve the opaque predicate, attempt to linearly decode the inline assembly data as executable logic, causing severe instruction desynchronization and extensive decoding failures across the subsequent basic blocks.

Because we inject inline assembly mid-compilation, our structural transformations are restricted strictly to additive instruction insertion, as we cannot natively delete or modify the surrounding machine code generated by the LLVM backend. Operating within these constraints, we implement stack frame diversification to manually manipulate the hardware stack and frame pointer registers via additive inline assembly.

```
// Stack frame diversification delta manipulating hardware registers
__asm__ volatile (
    "push %%rbp\n\t"
    "mov %%rsp, %%rbp\n\t"
    : : : "memory"
);
```

Figure 6.5: Stack Frame Diversification via Inline Assembly

This targeted manipulation generates heterogeneous, unpredictable stack layouts across the binary signature. We supplement these stack alterations by hardcoding low-level, explicit flow transfers that physically redirect the instruction pointer while remaining

strictly consistent with the higher-level IR logic, ensuring the overall execution flow does not derail.

We further obscure the binary footprint from static analysis by utilizing inline assembly to substitute standard C++ compilation outputs with explicit machine-code bytes. Rather than allowing the compiler backend to generate standard assembly mnemonics for programmatic operations, our strategy injects the exact binary encoding using data definition directives.

```
// Machine-code substitution replacing the Mov DX mnemonic
```

```
__asm__ volatile (".byte 0xBA" : : : "memory");
```

Figure 6.6: Machine-code Substitution via Data Directives

This explicitly forces the raw binary sequence 10111010b directly into the executable memory segment. By hardcoding the operation at the bit level, we fulfill the necessary computational state while completely masking the operational mnemonic from standard assembly heuristics and pattern-matching algorithms.

6.3 Dynamic API and NTAPI Loading

To completely eliminate a compiled binary's reliance on the static Import Address Table, the obfuscation architecture executes API call site tampering. This dynamic API resolution hides intended capabilities from static heuristics that parse the table for suspicious imports. Instead of allowing the compiler to natively link functions, the C++ code manually declares function pointers with strict signatures matching the target API. The program leverages OS loader functions, specifically `GetModuleHandleA` and `GetProcAddress`, to locate the target API dynamically within memory-loaded system libraries. Because resolution requires the exact API name as a string parameter, these strings are heavily encrypted in the source code to evade static string analysis and decrypted in memory strictly before execution.

```
// Delta Snippet: Dynamic API Resolution and String Decryption
```

```
typedef LPVOID(WINAPI* pfnVIRTUALALLOC)(HANDLE, LPVOID, SIZE_T,  
DWORD, DWORD);
```

```
// String decrypted dynamically in memory to evade static extraction
```

```
decrypt_string(enc_api_string);
```

```

pfnVIRTUALALLOCEx      dynamic_Alloc      =
(pfnVIRTUALALLOCEx)GetProcAddress(GetModuleHandleA("kernel32.dll"),
enc_api_string);

```

Figure 6.7: Dynamic API Resolution and String Decryption

To bypass behavioral monitors that place user-space hooks on standard kernel32.dll functions, the architecture shifts execution to the Native API layer residing within ntdll.dll. High-level APIs are mapped to their undocumented native equivalents, translating operations like VirtualAllocEx directly to NtAllocateVirtualMemory. Loading these native functions at runtime entirely omits them from the Import Address Table, circumventing endpoint detection systems that monitor standard library dependencies. Because Microsoft does not document these native APIs, the logic must manually implement complex structures and macros, utilizing InitializeObjectAttributes to configure memory allocation properties and defining explicit NTSTATUS error codes to handle kernel responses.

```

// Delta Snippet: NTAPI Structure Initialization and Manual Error Handling

#define STATUS_SUCCESS ((NTSTATUS)0x00000000L)

OBJECT_ATTRIBUTES obj_attr;

InitializeObjectAttributes(&obj_attr, NULL, 0, NULL, NULL);

// Execution routed through the dynamically loaded ntdll.dll pointer

NTSTATUS status = dynamic_NtAllocateVirtualMemory(process_handle,
&base_address, 0, &region_size, MEM_COMMIT, PAGE_EXECUTE_READWRITE);

```

Figure 6.8: NTAPI Structure Initialization and Manual Error Handling

Direct syscalls provide the ultimate evasion mechanism by completely bypassing the ntdll.dll API layer and its associated user-space hooks. Rather than executing the wrapped function provided by the operating system, the program interacts directly with the Windows kernel. This is achieved by embedding a custom assembly stub that relies on the exact System Service Number of the target kernel function, invoking the native syscall instruction to bypass hooked APIs entirely. Because these service numbers fluctuate across different Windows builds, development relies on generation tools like SysWhispers to automate the creation of tailored header and assembly files.

```

// Delta Snippet: Direct Syscall Assembly Stub

// Generated via: python.exe syswhispers.py -c mingw -f NtAllocateVirtualMemory -o
syscalls

__asm__ volatile (
    "mov r10, rcx\n\t"

    "mov eax, 0x18\n\t" // System Service Number (SSN) dynamically matched to target OS
    build

    "syscall\n\t"      // Direct transition to kernel execution

    "ret\n\t"
);

```

Figure 6.9: Direct Syscall Assembly Stub

When employing direct syscalls for tasks such as process enumeration via `NtQuerySystemInformation`, the C++ code must define advanced data structures like `SYSTEM_PROCESS_INFORMATION`. This allows the application to parse the low-level memory usage and parent-child identifiers returned directly by the kernel, maintaining full operational capability while remaining entirely invisible to user-mode behavioral sandboxes.

6.4 Self-Modifying Code (SMC) via XOR Decryption

Self-modifying code (SMC) architecture dynamically mutates its own physical instructions during runtime execution. We implement this mechanism to continuously overwrite the binary machine code resident in memory, rendering the execution sequence structurally unpredictable to static analysis algorithms. This shifting landscape heavily deters reverse engineering by generating the functional instructions explicitly at the moment of execution.

We execute these mutations utilizing Exclusive OR (XOR) bitwise obfuscation targeting the physical opcode bytes. The XOR operation merges the binary opcode string with a discrete key, flipping bits to generate an unrecognizable instruction signature. We utilize XOR masking due to its symmetric reversibility, allowing in-place memory mutation utilizing a unified key without disrupting the overall binary entropy, thereby evading heuristic analyzers targeting encrypted payloads.

To guarantee that adversaries cannot statically deduce the XOR key, our algorithm dynamically generates the cipher at runtime utilizing volatile hardware registers. The pipeline extracts an 8-bit key directly from the CPU's Stack Pointer (SP) register. Because the SP register continuously tracks the active stack state, the extracted value fluctuates non-deterministically during execution. The algorithm increments a tracking counter during stack manipulation. Upon reaching a predefined threshold, the key is regenerated from the active SP state. The injected runtime execution stub intercepts the target instruction, executes a byte-by-byte XOR operation utilizing the SP-derived key, and physically overwrites the original machine code in memory.

Modern operating systems deploy strict memory protections, such as Data Execution Prevention (DEP/W^X), marking standard .text code segments as strictly read-only to prevent malicious modification. Our architecture bypasses these constraints by executing the mutation through inline assembly and dynamic heap manipulation.

```
// Phase 4 Self-Modifying Code: Buffer Allocation bypassing DEP
#include <windows.h>

int main() {
    // Allocate memory with explicit Read/Write/Execute permissions
    void* exec_buffer = VirtualAlloc(NULL, 1024, MEM_COMMIT | MEM_RESERVE,
    PAGE_EXECUTE_READWRITE)

    // Copy the naked target function into the executable space
    memcpy(exec_buffer, &target_function, 1024);

    // Define a function pointer to the buffer and invoke execution
    void (*dynamic_func)() = (void (*)())exec_buffer;
    dynamic_func();

    // Deallocate and clean the execution environment
    VirtualFree(exec_buffer, 0, MEM_RELEASE);

    return 0;
}
```

Figure 6.10: DEP Bypass via Virtual Memory Allocation

The C++ pipeline declares the targeted function utilizing specific compiler qualifiers, such as `__declspec(naked)` or equivalent GCC attributes, preventing the compiler from generating standard prolog or epilog frames. This preserves exact instruction offsets. Because the code mutates within an abstract memory allocation, the inline assembly must dynamically locate its own Instruction Pointer (EIP). The engine executes a `CALL` instruction to push the active EIP to the stack, immediately executing a `POP` to retrieve the address into a working register. The pipeline calculates the exact memory offset of the targeted opcode and executes the mutation via a direct pointer write.

```
// Phase 4 SMC Inline Assembly: EIP Resolution and In-Place XOR Mutation

// The engine dynamically locates the instruction pointer and overwrites the opcode.

__asm__ volatile (
    "call get_eip\n\t"    // Push EIP to stack
    "get_eip:\n\t"
    "pop %%edx\n\t"      // Pop EIP into EDX register
    "add $15, %%edx\n\t"  // Calculate absolute offset to the target instruction
    "xorb $0x28, (%%edx)\n\t" // Execute in-place memory mutation via XOR key
    ::: "memory", "edx"
);
```

Figure 6.11: SMC Inline Assembly: EIP Resolution and In-Place XOR Mutation

This specific inline logic functionally alters the machine code at the exact moment of execution. For example, applying an XOR 0x28 mask against specific opcode bytes dynamically flips an `ADD` instruction into a `SUB` instruction, modifying the computational state before reversing the mutation on the subsequent iteration, securely finalizing the dynamic execution cycle.

7 Conclusions & Future Work

The implementation of the four-phase obfuscation architecture demonstrates that securing C++ applications against Man-At-The-End adversaries requires a cumulative, hybrid transformation pipeline. Relying on isolated obfuscation mechanisms allows automated analysis tools to normalize and bypass protections sequentially. By structurally intertwining source-level Abstract Syntax Tree modifications with low-level LLVM Intermediate Representation mutations, the architecture successfully blinds both static decompilation algorithms and dynamic symbolic execution engines.

The integration of advanced structural dismantling inherently degrades standard execution metrics. The Control Flow Flattening executed in Phase 2 drastically increases McCabe's Cyclomatic Complexity, resulting in a measurable expansion of the Abstract Syntax Tree node count and physical binary size. The implementation of Self-Modifying Code and dynamic Native API resolution in Phase 4 introduces runtime latency, as the CPU must continuously execute XOR decryptions and flush instruction cache pipelines. The architecture dictates a strict trade-off where maximum analytical obscurity directly penalizes temporal performance, requiring careful deployment targeting only the most critical cryptographic or licensing algorithms within an application.

The cumulative application of these four phases neutralizes modern reverse-engineering toolchains. The Phase 1 context hoisting and Phase 3 dead-code insertions successfully corrupt the graph-theoretic algorithms utilized by binary diffing engines. Furthermore, the integration of Type II Bi-Opaque predicates exploits the mathematical rigidity of Satisfiability Modulo Theories solvers. By forcing engines like KLEE and Triton to evaluate structurally ambiguous constraints and multi-level pointer aliasing, the architecture successfully induces path explosion and causes automated deobfuscators to erroneously prune legitimate execution paths. At the endpoint, the dynamic resolution of undocumented NTAPIs and the continuous runtime mutation of execution stubs actively evade behavioral sandboxes and signature-based heuristic monitors.

The deliberate destruction of the imperative execution graph severely complicates legitimate software maintenance and debugging operations. When the obfuscated binary triggers a legitimate runtime exception, the generated crash dumps and stack traces map exclusively to the flattened central dispatcher and the randomized identifier hashes. The original C++ hierarchical context is unrecoverable from the endpoint telemetry. Engineering teams must maintain isolated, heavily secured mapping databases linking the Phase 1 Unified Symbol Resolution hashes and Phase 2 state variables back to the baseline source code to facilitate post-deployment patching and vulnerability remediation.

Future iterations of this obfuscation framework must integrate dynamic environmental awareness directly into the execution pipeline. While the current architecture successfully mutates code to evade static signatures, executing Self-Modifying Code within isolated

analysis environments remains a detectable anomaly. Subsequent research will focus on embedding advanced anti-debugging constraints and hardware-level fingerprinting mechanisms directly into the Phase 3 opaque predicates. This evolution will allow the binary to autonomously verify the integrity of the host operating system, ensuring the metamorphic payloads solely decrypt and execute on verified, non-virtualized hardware.

REFERENCES

- [1] C. Cadar, D. Dunbar, and D. R. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI)*, 2008, pp. 209-224. Available: https://www.usenix.org/legacy/event/osdi08/tech/full_papers/cadar/cadar.pdf
- [2] C. Collberg, C. Thomborson, and D. Low, "A taxonomy of obfuscating transformations," Department of Computer Science, University of Auckland, Technical Report 148, 1997. Available: <http://profs.sci.univr.it/~giaco/download/Watermarking-Obfuscation/Obfuscation%20Taxonomy.pdf>
- [3] G. Jacob, H. Debar, and E. Filiol, "Behavioral detection of malware: From a survey towards an established taxonomy," *Journal in Computer Virology*, vol. 4, no. 3, pp. 251-266, 2008. Available: <https://doi.org/10.1007/s11416-008-0086-0>
- [4] P. Junod, J. Rinaldini, J. Wehrli, and J. Michielin, "Obfuscator-LLVM: Software protection for the masses," in *Proceedings of the 1st International Workshop on Software Protection*, 2015, pp. 3-9. Available: <https://doi.org/10.1109/SPRO.2015.10>
- [5] C. Linn and S. Debray, "Obfuscation of executable code to improve resistance to static disassembly," in *Proceedings of the 10th ACM Conference on Computer and Communications Security*, 2003, pp. 290-299. Available: <https://doi.org/10.1145/948109.948149>
- [6] M. E. Russinovich, D. A. Solomon, and A. Ionescu, *Windows Internals, Part 1: Covering Windows Server 2008 R2 and Windows 7*, 6th ed. Redmond, WA: Microsoft Press, 2012. Available: <https://learn.microsoft.com/en-us/sysinternals/resources/windows-internals>
- [7] M. Sharif, A. Lanzi, J. Giffin, and W. Lee, "Automatic reverse engineering of malware emulators," in *2009 30th IEEE Symposium on Security and Privacy*, 2009, pp. 94-109. Available: <https://doi.org/10.1109/sp.2009.27>
- [8] A. F. Tiu, "SysWhispers: Understanding Windows system calls," GitHub Repository, 2020. Available: <https://github.com/jthuraisamy/SysWhispers>

RESUME

Name Surname: Izzat Mammadzada

Date of Birth: 10.06.2005

Place of Birth: Baku, Azerbaijan

High School: Foreign Language-Oriented Republican Gymnasium named after S. Bahlulzada, 2022

Contact: izzatmammadzada@gmail.com

Internships:

DevOps Engineer Intern - Better Life Communication (BLC) - Istanbul - (02/2025 – 04/2025)

Software Developer Intern - Bluesense AI - Istanbul - (06/2025 – 09/2025)

Cybersecurity Intern - Bluesense AI - Istanbul - (09/2025 – 11/2025)

Penetration Tester Intern - Ema Security - Istanbul - (11/2025 – 04/2026)

Final Project Work: Advanced Code Obfuscation Techniques